Lab Assignment – Unit 4: Process and Thread Management in Unix/Linux

Course: System Programming in Unix/Linux

General Instructions

1. Use a Linux system (e.g., Ubuntu) with **gcc**, **pthread**, and standard developer tools installed.
2. To compile normal C programs:

   ```
   gcc filename.c -o output
   ```

3. To compile pthread-based programs:

   ```
   gcc filename.c -o output -lpthread
   ```

4. All programs must be well written
5. Include screenshots of execution outputs in your final lab submission.
6. For theory and analysis questions, write concise answers (2–4 sentences).

PART A: PROCESS MANAGEMENT

---

1. Process Identification & States

Objective:
Understand PID, PPID, and Linux process states such as Running (R), Sleeping (S), Zombie (Z), etc.

Tasks:

1. Run the following and identify your shell's PID and PPID:

   ```
   echo $$
   ps -u $USER
   ps -elf
   ```

2. Start a background process:

   ```
   sleep 500 &
   ```

Inspect its state using:

```
ps -elf | grep sleep
```

Terminate it using:

```
kill <PID>
```

3. Run `top`, then launch multiple:

   ```
   sha1sum /dev/zero &
   ```

   Observe CPU usage and load average.

4. Run:

   ```
   vmstat 1
   ```

   Observe **context switches (cs)** under system load.

---

2. Process Creation Using fork()

Program: `fork_demo.c`

Requirements:

- Create a child process using `fork()`.
- Child sleeps 2 seconds and exits with status **42**.
- Parent waits using `wait()` and prints exit status using `WIFEXITED()` and `WEXITSTATUS()`.

Analysis Questions:

1. What happens if `wait()` is omitted?
2. Why does `fork()` return twice?
3. How would you modify the program to create **three child processes**?

---

3. Program Replacement Using exec()

Program: `exec_ls.c`

Requirements:

- Parent forks a child.
- Child calls:

```
execl("/bin/ls", "ls", "-l", NULL);
```

- On failure, child prints error using `perror()` and exits using `_exit(1)`.

Analysis Questions:

1. Rewrite the same logic using `execvp()` with `char *argv[]`.
2. Why should the child use `_exit()` instead of `exit()` on exec failure?

---

4. Zombie Processes

Program: `zombie.c`

Requirements:

- Child exits immediately.
- Parent sleeps for 15 seconds.
- During this time, inspect zombie using:

```
ps -elf | grep defunct
```

Analysis Questions:

1. How can a parent avoid leaving zombie processes?
2. What resources does a zombie process still consume?

---

5. Orphan Processes

Program: `orphan.c`

Requirements:

- Parent exits immediately.
- Child sleeps, then prints its new PPID.

Analysis Questions:

1. What is the child's PPID before and after parent exit?
2. Which process reaps an orphaned child?

PART B: MULTITHREADING & SYNCHRONIZATION

---

6. Basic Thread Creation

Program: `thread_basic.c`

Requirements:

- Main creates a worker thread using `pthread_create()`.
- Worker prints thread ID using `pthread_self()`.
- Main uses `pthread_join()`.

Analysis Questions:

1. What happens if `pthread_join()` is omitted?
2. Is `pthread_t` the same as a process PID? Why or why not?

---

7. Passing Arguments to Threads

Program: `thread_args.c`

Requirements:

- Pass integer (e.g., 100) from main to thread using a `void*` argument.
- Thread casts and prints.

Analysis Questions:

1. Why is passing the address of a stack variable unsafe for detached threads?
2. How can you pass multiple arguments (e.g., `int id`, `char *name`)?

---

8. Demonstrating Race Conditions

Program: `race.c`

Requirements:

- Two threads increment a shared global counter 1,000,000 times.
- No synchronization.

- Final output is inconsistent (< 2,000,000).

Analysis Questions:

1. Why does `counter++` cause lost updates?
2. Identify the critical section.

---

9. Fixing Race Conditions Using Mutex

Program: `mutex_fix.c`

Requirements:

- Protect `counter++` using `pthread_mutex_lock()` and `pthread_mutex_unlock()`.
- Initialize and destroy mutex properly.

Analysis Questions:

1. What is the performance cost of fine-grained locking?
2. Suggest one technique to reduce lock contention.

---

10. Producer–Consumer Using Condition Variables

Program: `prod_cons.c`

Requirements:

- Implement a single-slot buffer.
- Producer generates 1 → 5.
- Consumer consumes each.
- Use `pthread_cond_wait()` (with mutex) and `pthread_cond_signal()`.

Analysis Questions:

1. Why must `pthread_cond_wait()` hold the mutex?
2. Why use a `while` loop instead of `if` around the condition?

---

11. Semaphore Demonstration

Program: `semaphore_demo.c`

Requirements:

- Use binary semaphore:
- `sem_init(&sem, 0, 1);`
- Two threads increment shared counter 100,000 times each.
- Final result must be exactly **200,000**.

Analysis Questions:

1. How to convert this to a counting semaphore with max 3 threads?
2. What is a key practical difference between mutex and binary semaphore?

---

12. Barriers

Program: `barrier.c`

Requirements:

- Three threads synchronize across three phases.
- Use `pthread_barrier_wait()`.
- Each prints "Phase N done" only after barrier synchronization.

Analysis Questions:

1. What if one thread crashes before reaching the barrier?
2. When is a barrier preferred over condition variables?

---

13. Inspecting Kernel Threads (NPTL)

Program: `show_tid.c`

Requirements:

- Use `syscall(SYS_gettid)` to print kernel TID.
- Verify using:

```
ps -T -p <main_pid>
```

Analysis Questions:

1. What does the NPTL 1:1 threading model mean?
2. How is it different from older M:N threading models?

PART C: THEORY QUESTIONS

Process Concepts

1. Explain the fork–exec model. Why are they separate?
2. What is Copy-on-Write (COW)? How does it optimize `fork()`?
3. Differentiate zombie and orphan processes and how the OS handles each.

Multitasking & Scheduling

1. Define context switch. What process state is saved?
2. Interpret load averages 3.2, 2.8, 2.0 on a 4-core system.
3. Why is preemptive multitasking preferred over cooperative multitasking?

Threads & Synchronization

1. Why must `pthread_cond_wait()` use a `while` loop? Explain spurious wakeups.
2. Describe the four Coffman conditions for deadlock.
3. Why are two condition variables (`not_full`, `not_empty`) used in producer–consumer?
4. Compare mutexes and binary semaphores. When is a semaphore more appropriate?